Nama: Khania Puji Auliya

NIM: 254107020236

Kelas: 1G

Prodi: D-IV Teknik Informatika

**LAPORAN JOBSHEET 7**

**PERCOBAAN 1**

- **Kode Program Mahasiswa15**

```
public class Mahasiswa15 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa15() {
    }

    public Mahasiswa15(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

- **Kode Program MahasiswaBerprestasi15**

```
public class MahasiswaBerprestasi15 {
    Mahasiswa15 [] listMhs = new Mahasiswa15[5];
    int idx;

    void tambah (Mahasiswa15 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa15 m:listMhs) {
            m.tampilInformasi();
            System.out.println("--------------------------------");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length-1; i++) {
            for (int j = 1; j < listMhs.length-i; j++) {
                if (listMhs[j].ipk > listMhs[j-1].ipk) {
                    Mahasiswa15 tmp = listMhs[j];
                    listMhs[j] = listMhs[j-1];
                    listMhs[j-1] = tmp;
                }
            }
        }
    }
```

```
void selectionSort() {
    for (int i = 0; i < listMhs.length-1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa15 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

void insertionSort() {
    for (int i = 1; i < listMhs.length; i ++) {
        Mahasiswa15 temp = listMhs[i];
        int j = i;
        while (j > 0 && listMhs[j-1].ipk < temp.ipk) {
            listMhs[j] = listMhs[j-1];
            j--;
        }
        listMhs[j] = temp;
    }
}

int sequentialSearching(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk==cari) {
            posisi=j;
```

```
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos) {
    if (pos!=-1) {
        System.out.println("Data mahasiswa dengan IPK:" + x + " ditemukan pada indeks
 " + pos);
    } else {
        System.out.println("Data " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos !=-1) {
        System.out.println("nim\t : " + listMhs[pos].nim);
        System.out.println("nama\t : " + listMhs[pos].nama);
        System.out.println("kelas\t : " + listMhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    } else {
        System.out.println("Data mahasiswa dengan IPK " + x + "tidak ditemukan");
    }
}
}
```

- **Kode Program MahasiswaDemo15**

```
import java.util.Scanner;

public class MahasiswaDemo15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
MahasiswaBerprestasi15 list = new MahasiswaBerprestasi15();
int jmlMhs = 5;

for (int i = 0; i < jmlMhs; i++) {
    System.out.println("Masukkan data mahasiswa ke-" + (i + 1));
    System.out.print("NIM   : ");
    String nim = sc.nextLine();
    System.out.print("Nama  : ");
    String nama = sc.nextLine();
    System.out.print("Kelas : ");
    String kelas = sc.nextLine();
    System.out.print("IPK   : ");
    double ipk = sc.nextDouble();
    sc.nextLine();
    System.out.println("--------------------------------");

    Mahasiswa15 m = new Mahasiswa15(nim, nama, kelas, ipk);
    list.tambah(m);
}

list.tampil();
//melakukan pencarian data sequential
System.out.println("---------------------------------------------");
System.out.println("Pencarian data");
System.out.println("---------------------------------------------");
System.out.println("Masukkan ipk mahasiswa yang dicari: ");
System.out.print("IPK: ");
double cari = sc.nextDouble();

System.out.println("menggunakan sequential searching");
double posisi = list.sequentialSearching(cari);
int pss = (int) posisi;
```

```
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);

        System.out.println("Data Mahasiswa sebelum sorting: ");
        list.tampil();

        System.out.println("Data yang sudah terurut menggunakan INSERTION SORT
    (DSC)");
        list.insertionSort();
        list.tampil();
    }
}
```

- **Hasil Running**

```
Masukkan data mahasiswa ke-1
NIM   : 111
Nama  : Adi
Kelas : 2
IPK   : 3,6
--------------------------------
Masukkan data mahasiswa ke-2
NIM   : 222
Nama  : Tio
Kelas : 2
IPK   : 3,8
--------------------------------
Masukkan data mahasiswa ke-3
NIM   : 333
Nama  : Ila
Kelas : 2
IPK   : 3,0
--------------------------------
Masukkan data mahasiswa ke-4
NIM   : 444
Nama  : Lia
Kelas : 2
IPK   : 3,5
--------------------------------
Masukkan data mahasiswa ke-5
NIM   : 555
```

```
Nama  : Fia
Kelas : 2
IPK   : 3,3
--------------------------------
Nama: Adi
NIM: 111
Kelas: 2
IPK: 3.6
---------------------------------
Nama: Tio
NIM: 222
Kelas: 2
IPK: 3.8
---------------------------------
Nama: Ila
NIM: 333
Kelas: 2
IPK: 3.0
---------------------------------
Nama: Lia
NIM: 444
Kelas: 2
IPK: 3.5
---------------------------------
Nama: Fia
NIM: 555
Kelas: 2
IPK: 3.3
---------------------------------
---------------------------------------------
Pencarian data
---------------------------------------------
Masukkan ipk mahasiswa yang dicari:
IPK: 3,5
menggunakan sequential searching
Data mahasiswa dengan IPK:3.5 ditemukan pada indeks 3
nim      : 444
nama     : Lia
kelas    : 2
ipk      : 3.5
```

**Pertanyaan!**

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!
2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){
    posisi=j;
    break;
}
```

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.
5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

**Jawaban**

1. tampilDataSearch menampilkan detail lengkap data mahasiswa (NIM, nama, kelas, IPK), sedangkan tampilPosisi menampilkan posisi/indeks ditemukannya data.
2. break berfungsi untuk menghentikan loop for secara paksa segera setelah data yang dicari ditemukan. Begitu IPK cocok ditemukan di indeks j, nilai posisi disimpan lalu perulangan langsung berhenti, tidak melanjutkan pengecekan elemen berikutnya.
3. Variabel pos berfungsi sebagai penanda lokasi atau indeks dari data yang berhasil ditemukan di dalam array listMhs.
4. Apabila terdapat lebih dari satu data mahasiswa yang memiliki nilai IPK sama, maka program akan menampilkan data yang pertama kali ditemukan, yaitu data dengan indeks terkecil dalam array.
5. Jika perintah break dihapus dari kode tersebut, maka loop akan terus berjalan hingga seluruh elemen array selesai diperiksa meskipun data yang dicari sudah ditemukan sebelumnya.

**PERCOBAAN 2**

Kode program mahasiswa15 sama dengan percobaan 1

- **Kode Program MahasiswaBerprestasi15**

```
public class MahasiswaBerprestasi15 {
    Mahasiswa15 [] listMhs = new Mahasiswa15[5];
    int idx;
    void tambah (Mahasiswa15 m) {
```

```
    if (idx < listMhs.length) {
        listMhs[idx] = m;
        idx++;
    } else {
        System.out.println("Data sudah penuh");
    }
}

void tampil() {
    for (Mahasiswa15 m:listMhs) {
        m.tampilInformasi();
        System.out.println("--------------------------------");
    }
}

void bubbleSort() {
    for (int i = 0; i < listMhs.length-1; i++) {
        for (int j = 1; j < listMhs.length-i; j++) {
            if (listMhs[j].ipk > listMhs[j-1].ipk) {
                Mahasiswa15 tmp = listMhs[j];
                listMhs[j] = listMhs[j-1];
                listMhs[j-1] = tmp;
            }
        }
    }
}

void selectionSort() {
    for (int i = 0; i < listMhs.length-1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa15 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

void insertionSort() {
    for (int i = 1; i < listMhs.length; i ++) {
        Mahasiswa15 temp = listMhs[i];
        int j = i;
        while (j > 0 && listMhs[j-1].ipk < temp.ipk) {
            listMhs[j] = listMhs[j-1];
            j--;
        }
        listMhs[j] = temp;
```

```
        }
    }

    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int j = 0; j < listMhs.length; j++) {
            if (listMhs[j].ipk==cari) {
                posisi=j;
                break;
            }
        }
        return posisi;
    }

    void tampilPosisi(double x, int pos) {
        if (pos!=-1) {
            System.out.println("Data mahasiswa dengan IPK:" + x + " ditemukan pada indeks
" + pos);
        } else {
            System.out.println("Data " + x + " tidak ditemukan");
        }
    }

    void tampilDataSearch(double x, int pos) {
        if (pos !=-1) {
            System.out.println("nim\t : " + listMhs[pos].nim);
            System.out.println("nama\t : " + listMhs[pos].nama);
            System.out.println("kelas\t : " + listMhs[pos].kelas);
            System.out.println("ipk\t : " + x);
        } else {
            System.out.println("Data mahasiswa dengan IPK " + x + "tidak ditemukan");
        }
    }

    int findBinarySearch(double cari, int left, int right) {
        int mid;
        if (right>=left) {
            mid = (left + right)/2;
            if (cari == listMhs[mid].ipk) {
                return (mid);
            }
            else if (listMhs[mid].ipk>cari) {
                return findBinarySearch(cari, left, mid-1);
            } else {
                return findBinarySearch(cari, mid+1, right);
            }
        }
        return -1;
    }
}
```

- **Kode Program MahasiswaDemo15**

```
import java.util.Scanner;
public class MahasiswaDemo15 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      MahasiswaBerprestasi15 list = new MahasiswaBerprestasi15();
      int jmlMhs = 5;

      for (int i = 0; i < jmlMhs; i++) {
         System.out.println("Masukkan data mahasiswa ke-" + (i + 1));
         System.out.print("NIM   : ");
         String nim = sc.nextLine();
         System.out.print("Nama  : ");
         String nama = sc.nextLine();
         System.out.print("Kelas : ");
         String kelas = sc.nextLine();
         System.out.print("IPK   : ");
         double ipk = sc.nextDouble();
         sc.nextLine();
         System.out.println("-------------------------------");

         Mahasiswa15 m = new Mahasiswa15(nim, nama, kelas, ipk);
         list.tambah(m);
      }

      list.tampil();
      //melakukan pencarian data binary
      System.out.println("---------------------------------------------");
      System.out.println("Pencarian data");
      System.out.println("---------------------------------------------");
      System.out.println("Masukkan ipk mahasiswa yang dicari: ");
      System.out.print("IPK: ");
      double cari = sc.nextDouble();
      System.out.println("----------------------------------");
      System.out.println("menggunakan binary searching");
      System.out.println("----------------------------------");
      double posisi2 = list.findBinarySearch(cari, 0, jmlMhs-1);
      int pss2 = (int)posisi2;
      list.tampilPosisi(cari, pss2);
      list.tampilDataSearch(cari, pss2);

      System.out.println("Data Mahasiswa sebelum sorting: ");
      list.tampil();

      System.out.println("Data yang sudah terurut menggunakan INSERTION SORT
(DSC)");
      list.insertionSort();
      list.tampil();
   }
}
```

- **Hasil Running**

```
Masukkan data mahasiswa ke-1
NIM   : 111
Nama  : adi
Kelas : 2
IPK   : 3,1
--------------------------------
Masukkan data mahasiswa ke-2
NIM   : 222
Nama  : ila
Kelas : 2
IPK   : 3,2
--------------------------------
Masukkan data mahasiswa ke-3
NIM   : 333
Nama  : lia
Kelas : 2
IPK   : 3,3
--------------------------------
Masukkan data mahasiswa ke-4
NIM   : 444
Nama  : susi
Kelas : 2
IPK   : 3,5
--------------------------------
Masukkan data mahasiswa ke-5
NIM   : 555
Nama  : anita
Kelas : 2
IPK   : 3,7
--------------------------------
Nama: adi
NIM: 111
Kelas: 2
IPK: 3.1
---------------------------------
Nama: ila
NIM: 222
Kelas: 2
IPK: 3.2
---------------------------------
Nama: lia
NIM: 333
Kelas: 2
IPK: 3.3
--------------------------------
Nama: susi
NIM: 444
Kelas: 2
```

```
IPK: 3.5
--------------------------------
Nama: anita
NIM: 555
Kelas: 2
IPK: 3.7
--------------------------------
---------------------------------------------
Pencarian data
---------------------------------------------
Masukkan ipk mahasiswa yang dicari:
IPK: 3,7
----------------------------------
menggunakan binary searching
----------------------------------
Data mahasiswa dengan IPK:3.7 ditemukan pada indeks 4
nim      : 555
nama     : anita
kelas    : 2
ipk      : 3.7
```

**Pertanyaan!**

1. Tunjukkan pada kode program yang mana proses divide dijalankan!
2. Tunjukkan pada kode program yang mana proses conquer dijalankan!
3. Apa fungsi left, right, dan mid?
4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?
5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai
6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

**Jawaban**

1. ```
   mid = (left + right)/2;
   ```
   Pada baris ini, array dibagi menjadi dua bagian dengan cara menghitung indeks tengah (mid) dari rentang yang sedang diperiksa.
   ```

2. 
```
else if (listMhs[mid].ipk>cari) {
    return findBinarySearch(cari, left, mid-1);
} else {
    return findBinarySearch(cari, mid+1, right);
}
```
3. left adalah batas indeks paling kiri dari rentang pencarian

   right adalah batas indeks paling kanan

   mid adalah indeks tengah yang dihitung dari (left + right) / 2.
4. Program tetap berjalan tanpa error, namun hasil pencariannya bisa salah atau tidak akurat. Binary search mensyaratkan data harus terurut terlebih dahulu, karena logika perbandingan di mid tidak dapat diandalkan jika data acak.
5. 
```
else if (listMhs[mid].ipk<cari) {
    return findBinarySearch(cari, left, mid-1);
} else {
    return findBinarySearch(cari, mid+1, right);
}
```

   kode program yang diubah, yang awalnya lebih dari (>) diubah menjadi kurang dari (<).

```
Masukkan data mahasiswa ke-1
NIM   : 111
Nama  : adi
Kelas : 2
IPK   : 3,8
--------------------------------
Masukkan data mahasiswa ke-2
NIM   : 222
Nama  : ila
Kelas : 2
IPK   : 3,7
--------------------------------
Masukkan data mahasiswa ke-3
NIM   : 333
Nama  : lia
Kelas : 2
IPK   : 3,5
--------------------------------
Masukkan data mahasiswa ke-4
NIM   : 444
Nama  : susi
Kelas : 2
IPK   : 3,4
--------------------------------
Masukkan data mahasiswa ke-5
NIM   : 555
Nama  : anita
```

```
Kelas : 2
IPK   : 3,2
--------------------------------
Nama: adi
NIM: 111
Kelas: 2
IPK: 3.8
--------------------------------
Nama: ila
NIM: 222
Kelas: 2
IPK: 3.7
--------------------------------
Nama: lia
NIM: 333
Kelas: 2
IPK: 3.5
--------------------------------
Nama: susi
NIM: 444
Kelas: 2
IPK: 3.4
--------------------------------
Nama: anita
NIM: 555
Kelas: 2
IPK: 3.2
--------------------------------
---------------------------------------------
Pencarian data
---------------------------------------------
Masukkan ipk mahasiswa yang dicari:
IPK: 3,2
-----------------------------------
menggunakan binary searching
-----------------------------------
Data mahasiswa dengan IPK:3.2 ditemukan pada indeks 4
nim      : 555
nama     : anita
kelas    : 2
ipk      : 3.2
```

6. Ketika right menjadi lebih kecil dari left, kondisi if (right >= left) menjadi false, artinya seluruh rentang sudah diperiksa. Program kemudian mengeksekusi return -1 sebagai tanda data tidak ditemukan.
7. **Penambahan kode program MahasiswaBerprestasi15**

```
Mahasiswa15 [] listMhs;

  MahasiswaBerprestasi15(int jumlah) {
     listMhs = new Mahasiswa15[jumlah];
  }
```

**Penambahan kode program MahasiswaDemo15**

```
System.out.print("Masukkan jumlah mahasiswa: ");
  int jmlMhs = sc.nextInt();
  sc.nextLine();
  MahasiswaBerprestasi15 list = new MahasiswaBerprestasi15(jmlMhs);
```

**Hasil Running**

```
Masukkan jumlah mahasiswa: 3
Masukkan data mahasiswa ke-1
NIM   : 111
Nama  : adi
Kelas : 2
IPK   : 3,8
-------------------------------
Masukkan data mahasiswa ke-2
NIM   : 222
Nama  : lia
Kelas : 2
IPK   : 3,6
-------------------------------
Masukkan data mahasiswa ke-3
NIM   : 333
Nama  : ali
Kelas : 2
IPK   : 3,5
-------------------------------
Nama: adi
NIM: 111
Kelas: 2
IPK: 3.8
--------------------------------
Nama: lia
NIM: 222
Kelas: 2
```

```
IPK: 3.6
--------------------------------
Nama: ali
NIM: 333
Kelas: 2
IPK: 3.5
--------------------------------
---------------------------------------------
Pencarian data
---------------------------------------------
Masukkan ipk mahasiswa yang dicari:
IPK: 3,5
-----------------------------------
menggunakan binary searching
-----------------------------------
Data mahasiswa dengan IPK:3.5 ditemukan pada indeks 2
nim      : 333
nama     : ali
kelas    : 2
ipk      : 3.5
```